A Reproducible Empirical Comparison of SAT Encodings and Exact Solvers for the N-Queens Problem

Nguyen Anh Son

University of Engineering and Technology, Vietnam National University, Hanoi

ARTICLE INFO

Keywords:
N-Queens
Boolean Satisfiability
Constraint Programming
Mixed Integer Programming
CDCL Heuristics
Empirical Benchmarking

ABSTRACT

The N-Queens problem is a classic combinatorial benchmark widely used to evaluate algorithmic efficiency in artificial intelligence. This study presents a rigorous empirical comparison of exact solving paradigms—Boolean Satisfiability (SAT), Constraint Programming (CP), and Mixed Integer Programming (MIP). We evaluate five Boolean SAT encodings alongside four exact solvers within a strict, subprocess-isolated, deterministically seeded benchmarking framework. Our primary full benchmark covers up to $N = 100$ (495 planned official trials), supplemented by a large-scale experiment at $N = 200$ (45 planned trials).

Our results demonstrate that structural compactness in SAT encodings is a poor predictor of Conflict-Driven Clause Learning (CDCL) resolution time; the SAT-Sequential encoding generated the second-fewest clauses but timed out at $N = 200$, whereas heavily parameterized SAT formulations resolved much faster. At $N = 200$, among the four configurations that solved successfully, SAT-Commander achieved the lowest median pipeline runtime (0.441 s) and lowest peak sampled process-tree RSS (146.2 MiB), whereas SAT-Pairwise was proactively excluded by a preflight memory policy ($>$ 4 GiB projected for 13.2 million clauses vs. 798.34 MiB available cap). Notably, the SAT-Product encoding exhibited extreme non-monotonic search pathology, timing out at intermediate problem sizes ($N \in \{40, 44, 50, 64\}$) while resolving $N = 100$ and $N = 200$ successfully. Constraint Programming offers a compact high-level formulation using N integer decision variables and global constraints, avoiding the need to explicitly construct the SAT CNF encodings evaluated in this study. However, the observed memory consumption depends on the solver implementation and does not consistently favor CP across all tested configurations. At $N = 200$, IBM CP Optimizer's Community Edition hit a hard search-space license limit (2^{1000}), emphasizing that both algorithmic complexity and software environment restrictions dictate practical large-scale solvability. This paper provides a fully reproducible dataset and quantitative insights into the practical trade-offs of constraint modeling techniques.

1. Introduction

The Constraint Satisfaction Problem (CSP) framework is foundational to modeling complex combinatorial search and placement problems in computer science and artificial intelligence (Russell and Norvig, 2010). Within this domain, the N-Queens problem—the challenge of placing N non-attacking queens on an $N \times N$ chessboard—serves as a canonical benchmark for evaluating the efficiency and scalability of exact search algorithms (Bell and Stevens, 2009). While standard N-Queens on an unconstrained board admits constructive linear-time $O(N)$ solutions, it serves as an invaluable model for assessing how general-purpose reasoning engines handle expanding symmetric constraints without problem-tailored shortcuts (distinguishing it from the generalized N-Queens Completion decision problem, which is NP-complete (Gent, Jefferson and Nightingale, 2017)). As the problem size N increases, naive enumeration becomes computationally intractable, necessitating sophisticated exact solving paradigms.

Currently, exact solvers rely on three primary modeling paradigms: Boolean Satisfiability (SAT), Constraint Programming (CP), and Mixed Integer Programming (MIP) (Gomes, Kautz, Sabharwal and Selman, 2008; Rossi, Van Beek and Walsh, 2006; Wolsey, 2020). In the Boolean SAT paradigm, high-level structural constraints must be transformed into Conjunctive Normal Form (CNF) via variable-heavy At-Most-One (AMO) encodings. Different encoding strategies—such as Pairwise, Binary, Sequential Counter, Commander, and Product—drastically alter the size and structure of the resulting CNF formula (Sinz, 2005; Frisch and Giannaros, 2010). Conversely, native Constraint Programming implementations maintain the problem's global structural semantics by directly applying constraints over integer variables (Régin, 1994), while Mixed Integer Programming leverages linear inequality relaxations over binary decision bounds.

A recurring assumption in SAT modeling is that structural complexity—specifically the total number of generated clauses or auxiliary variables—correlates directly with the time required by Conflict-Driven Clause Learning (CDCL) solvers to find a solution. However, real-world benchmarking often reveals non-linear scaling behaviors heavily influenced by the solver's internal heuristics (Marques-Silva, Lynce and Malik, 2009). Furthermore, while algorithmic complexity theoretically bounds performance, the practical runtime characteristics across the SAT, CP, and MIP paradigms on the exact same problem formulations remain sparsely compared in unified experimental setups.

The objective of this research is to execute a strictly controlled empirical comparison of these exact solving techniques on the N-Queens problem up to $N = 100$. Specifically, we address the following research questions: 1. How do different AMO encodings affect the structural complexity of SAT formulations? 2. How does the choice of SAT encoding impact solving time and completion reliability under a uniform CDCL configuration? 3. How do general-purpose Boolean reasoning frameworks compare against specialized native CP and MIP solvers? 4. What are the practical scalability limitations of these paradigms?

To answer these questions, extending earlier open baseline explorations (Duong, 2024) with expanded solvers, memory supervision, and rigorous multi-trial descriptive profiling, we constructed an automated, subprocess-isolated benchmarking framework. We evaluate five SAT encodings solved via PySAT Glucose3, against OR-Tools CP-SAT, IBM CP Optimizer, Gurobi MIP, and IBM CPLEX MIP.

The primary contributions of this paper are:

- A comprehensive, reproducible empirical dataset reporting 540 scheduled experimental records (495 at $N \leq 100$ and 45 at $N = 200$) across 9 distinct exact solving methodologies, including 450 successfully validated solutions and explicitly documented timeout, licensing, and resource-policy outcomes.
- Evidence that structural compactness (clause count minimization) is a poor predictor of CDCL resolution time on placement constraints.
- The identification of severe non-monotonic search pathologies in specific structural encodings (e.g., the SAT-Product encoding) under standard phase-selection heuristics.
- A direct comparative baseline characterizing the trade-offs between Constraint Programming formulations and Boolean SAT encodings, showing that well-engineered SAT encodings (such as Binary and Commander) remain exceptionally competitive at scale.

2. Background and Related Work

2.1. Propositional Satisfiability and CDCL

Propositional Satisfiability (SAT) is the problem of determining if there exists an assignment of boolean values to variables that satisfies a given formula. The Boolean satisfiability problem was the first known NP-complete problem (Cook, 1971). Most modern SAT solvers require the input to be in Conjunctive Normal Form (CNF), meaning the formula is a conjunction of clauses, where each clause is a disjunction of literals. Modern exact SAT solving is dominated by Conflict-Driven Clause Learning (CDCL) algorithms (Marques-Silva et al., 2009), which extend the classic DPLL backtracking search by inferring and learning new conflict clauses to prune the search space dynamically. Solvers such as Glucose heavily optimize this process by aggressively managing the learned clause database based on Literal Block Distance (LBD) (Audemard and Simon, 2009).

2.2. At-Most-One (AMO) Encodings

Translating cardinality constraints—such as "at most one variable among a set can be true"—into CNF is a critical aspect of SAT modeling. The naive **Pairwise** (or Binomial) encoding operates without auxiliary variables by generating an exclusion clause for every possible pair, requiring $\Theta(m^2)$ clauses for a group of m variables, which scales to $\Theta(N^3)$ clauses across the full N-Queens board (Frisch and Giannaros, 2010).

To mitigate this cubic clause explosion, auxiliary variables are introduced. The **Binary** (or Bitwise) encoding maps the m variables to $\lceil \log_2 m \rceil$ auxiliary bits, generating $\Theta(m \log m)$ clauses per group (Frisch and Giannaros, 2010) and yielding $\Theta(N^2 \log N)$ total clauses across the board. The **Sequential Counter** encoding by Sinz (2005) mimics a unary adder, bounding both auxiliary variables and clauses to $O(m)$ per group ($4N^2 - 6N + 2$ auxiliary variables and $\Theta(N^2)$ clauses total). The **Commander** encoding (Klieber and Kwon, 2007) recursively partitions the original variables into small groups (here evaluated with recursive group size $k = 3$), electing a "commander" variable for each group and

generating $O(m)$ auxiliary variables and clauses per group ($\Theta(N^2)$ clauses total). Finally, the **2-Product** encoding structures variables onto a 2D grid, translating the 1D AMO constraint into row and column constraints using $O(\sqrt{m})$ auxiliary variables and $O(m)$ clauses per group (Chen, 2010), likewise achieving a total formula complexity of $\Theta(N^2)$ clauses across all rows, columns, and diagonals of the board.

2.3. Constraint Programming and Mixed Integer Programming

Constraint Programming (CP) provides a higher-level abstraction than SAT by operating directly on integer domains (Rossi et al., 2006). Central to CP is the `AllDifferent` global constraint, which ensures that all variables in a given set take distinct values. Efficient domain filtering algorithms, notably those based on bipartite matching (Régin, 1994), actively prune the integer domains during search without needing explicit upfront Boolean expansion. Solvers like IBM CP Optimizer leverage traditional integer domain propagation directly. In contrast, modern hybrid frameworks such as OR-Tools CP-SAT (Perron and Furnon, 2019) employ Lazy Clause Generation (LCG), coupling high-level constraint propagation with an underlying CDCL SAT solving core that dynamically generates explanation clauses during search. Mixed Integer Programming (MIP) relies on linear inequalities over continuous and integer variables (Wolsey, 2020). While powerful for optimization problems, applying MIP solvers like Gurobi or CPLEX to pure feasibility problems like N-Queens translates logical implications into algebraic constraints bounded by linear programming relaxations.

3. Methodology

3.1. Mathematical Definition of N-Queens

The N-Queens problem requires placing N queens on an $N \times N$ chessboard such that no two queens attack each other. In the Boolean paradigm, the problem is modeled using N^2 primary decision variables $x_{i,j} \in \{0, 1\}$, where:

$$x_{i,j} = \begin{cases} 1 & \text{if a queen occupies row } i \text{ and column } j \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

for $0 \leq i, j < N$.

This formulation strictly enforces four directional constraints:

- **Row Constraints:** Exactly one queen per row.

$$\sum_{j=0}^{N-1} x_{i,j} = 1 \quad \forall i \in \{0, \dots, N-1\} \tag{2}$$

- **Column Constraints:** Exactly one queen per column.

$$\sum_{i=0}^{N-1} x_{i,j} = 1 \quad \forall j \in \{0, \dots, N-1\} \tag{3}$$

- **Main Diagonal Constraints:** At most one queen on any main diagonal. Let $d = i - j$. For each $d \in \{-(N-2), \dots, N-2\}$:

$$\sum_{i-j=d} x_{i,j} \leq 1 \tag{4}$$

- **Anti-Diagonal Constraints:** At most one queen on any anti-diagonal. Let $s = i+j$. For each $s \in \{1, \dots, 2N-3\}$:

$$\sum_{i+j=s} x_{i,j} \leq 1 \tag{5}$$

Note that diagonals with only a single cell ($d \in \{-(N-1), N-1\}$, $s \in \{0, 2N-2\}$) trivially satisfy the at-most-one condition and do not require explicit constraints.

3.2. Boolean SAT Constraints and Encodings

In Boolean Satisfiability, an *Exactly-One* (EO) constraint is represented as the logical conjunction of an *At-Least-One* (ALO) clause and an *At-Most-One* (AMO) constraint:

$$EO(X) = ALO(X) \wedge AMO(X) \tag{6}$$

For a set of boolean variables $X = \{x_1, \dots, x_m\}$, the ALO constraint is simply a single disjunction:

$$ALO(x_1, \dots, x_m) = \bigvee_{k=1}^{m} x_k \tag{7}$$

The AMO constraint, however, can be encoded into Conjunctive Normal Form (CNF) using various strategies. Formally, an encoding of an At-Most-One constraint over primary literals $X = \{x_1, \dots, x_m\}$ introduces a set of auxiliary variables $Y = \{y_1, \dots, y_k\}$ and a clause set $E(X, Y)$ satisfying the logical equivalence:

$$\text{AMO}(X) \iff \exists Y.\, E(X, Y) \tag{8}$$

Specifically, for any truth assignment τ on X, $\sum_{i=1}^{m} \tau(x_i) \leq 1$ if and only if there exists a truth assignment τ' on Y such that $\tau \cup \tau'$ satisfies every clause in $E(X, Y)$.

Our framework implements five distinct AMO encoding strategies:

1. **Pairwise (Binomial):** Asserts that no two literals in X can simultaneously be true without adding auxiliary variables ($Y = \emptyset$):

$$E_{\text{Pairwise}}(X) = \bigwedge_{1 \leq a < b \leq m} (\neg x_a \vee \neg x_b) \tag{9}$$

   Generates $\binom{m}{2} = \frac{m(m-1)}{2}$ binary clauses per group ($\Theta(m^2)$). Across all $6N - 6$ constraint groups on the $N \times N$ chessboard, this yields a cubic total clause complexity.

$$C_{\text{pairwise}}(N) = 2N + N^2(N-1) + \frac{N(N-1)(2N-1)}{3} \in \Theta(N^3) \tag{10}$$

2. **Binary (Bitwise):** Introduces $k = \lceil \log_2 m \rceil$ auxiliary variables $Y = \{b_1, \dots, b_k\}$. Each literal x_i ($1 \leq i \leq m$) is assigned a unique binary bitstring representation $\text{bin}(i-1) = (c_{i,1}, \dots, c_{i,k}) \in \{0,1\}^k$. Setting $x_i = 1$ forces the auxiliary variables to match this representation:

$$E_{\text{Binary}}(X, Y) = \bigwedge_{i=1}^{m} \bigwedge_{j=1}^{k} \left(\neg x_i \vee l_{i,j}\right), \quad \text{where } l_{i,j} = \begin{cases} b_j & \text{if } c_{i,j} = 1 \\ \neg b_j & \text{if } c_{i,j} = 0 \end{cases} \tag{11}$$

   This produces $\lceil \log_2 m \rceil$ auxiliary variables and $m\lceil \log_2 m \rceil \in \Theta(m \log m)$ binary clauses per group. Summed over all board rows, columns, and diagonals, the whole-board clause complexity is $\Theta(N^2 \log N)$.

3. **Sequential Counter:** Implements Sinz's unary counter (Sinz, 2005) using $m - 1$ auxiliary variables $Y = \{s_1, \dots, s_{m-1}\}$. The variable s_i indicates whether at least one variable in the prefix $\{x_1, \dots, x_i\}$ is true:

$$\begin{aligned} E_{\text{Sequential}}(X, Y) = {} & (\neg x_1 \vee s_1) \wedge (\neg x_m \vee \neg s_{m-1}) \\ & \wedge \bigwedge_{i=2}^{m-1} \Big((\neg x_i \vee s_i) \wedge (\neg s_{i-1} \vee s_i) \wedge (\neg x_i \vee \neg s_{i-1}) \Big) \end{aligned} \tag{12}$$

   Generates $m - 1$ auxiliary variables and $3m - 4$ clauses for each group of size $m \geq 2$. Across the full board, the exact totals are:

$$A_{\text{sequential}}(N) = 4N^2 - 6N + 2 \in \Theta(N^2) \tag{13}$$

$$C_{\text{sequential}}(N) = 12N^2 - 22N + 12 \in \Theta(N^2) \tag{14}$$

4. **Commander:** Recursively partitions literals into subgroups of size at most $k = 3$ (Klieber and Kwon, 2007). For each subgroup $G_j \subseteq X$ ($|G_j| \leq 3$), a commander variable c_j is introduced, local pairwise exclusion is enforced, and truth propagation is defined via implications:

$$E_{\text{Commander_group}}(G_j, c_j) = \left(\bigwedge_{g_a, g_b \in G_j, a<b} (\neg g_a \vee \neg g_b) \right) \wedge \left(\bigwedge_{g \in G_j} (\neg g \vee c_j) \right) \tag{15}$$

   The set of commander variables $\{c_j\}$ is then recursively constrained by AMO until a single variable remains. This requires $O(m)$ auxiliary variables and clauses per group, yielding $\Theta(N^2)$ clauses across the board.
5. **2-Product:** Structures literals onto a 2D grid of dimensions $p \times q$ where $p = \lceil \sqrt{m} \rceil$ and $q = \lceil m/p \rceil$ (Chen, 2010). Introducing auxiliary row variables $U = \{u_1, \ldots, u_p\}$ and column variables $V = \{v_1, \ldots, v_q\}$, each literal x_k at grid coordinates (r, c) implies its respective row and column bits:

$$E_{\text{Product}}(X, U, V) = \bigwedge_{k=1}^{m} \Big((\neg x_k \vee u_{r(k)}) \wedge (\neg x_k \vee v_{c(k)}) \Big) \wedge E_{\text{Pairwise}}(U) \wedge E_{\text{Pairwise}}(V) \tag{16}$$

   Requiring $p + q \in O(\sqrt{m})$ auxiliary variables and $2m + \binom{p}{2} + \binom{q}{2} \in O(m)$ clauses per group, this encoding scales to $\Theta(N^2)$ clauses across the board.

The formal inductive proofs for the exact closed-form expressions (10) and for Sinz's sequential counter are presented in Appendix A, verified for $N = 4$ ($C_{\text{pairwise}} = 84, C_{\text{sequential}} = 116$) and $N = 100$ ($C_{\text{pairwise}} = 1,646,900, C_{\text{sequential}} = 117,812$).

All SAT formulations are generated via the Python API and passed to the Glucose3 solver (Audemard and Simon, 2009) via the PySAT library. To ensure strict isolation of structural efficiency from heuristic tuning, the `solver_default` phase policy was enforced.

3.3. Comparison of Mathematical Formulations and Solving Paradigms

Our evaluation directly compares three fundamentally different exact solving paradigms applied to the exact same instance sizes of the N-Queens problem.

Boolean SAT: The problem is modeled with N^2 primary Boolean variables and purely logical CNF constraints. We evaluate the five AMO encodings (Pairwise, Binary, Sequential, Commander, Product), all solved via the CDCL-based Glucose3 solver with a common phase policy.

Constraint Programming (CP): CP frameworks represent the problem using N integer decision variables $q_i \in \{0, \ldots, N-1\}$ (representing the column of the queen in row i). The placement constraints are modeled as three global `AllDifferent` constraints (Régin, 1994):

$$\text{AllDifferent}(q_0, \ldots, q_{N-1}) \tag{17}$$

$$\text{AllDifferent}(q_0 - 0, \ldots, q_{N-1} - (N-1)) \tag{18}$$

$$\text{AllDifferent}(q_0 + 0, \ldots, q_{N-1} + (N-1)) \tag{19}$$

This formulation was solved independently using OR-Tools CP-SAT and IBM CP Optimizer. While the input formulation is identical, the underlying solving architectures differ fundamentally: IBM CP Optimizer relies on classical CP integer domain filtering, whereas OR-Tools CP-SAT operates as a Lazy Clause Generation (LCG) solver, combining CP propagation with an underlying CDCL Boolean satisfiability core that dynamically derives explanation clauses.

Mixed Integer Programming (ILP/MIP): Using the same N^2 binary decision variables $x_{i,j} \in \{0, 1\}$ as the SAT model, the MIP formulation translates the logical constraints into continuous linear equality and inequality constraints. For $N \geq 2$, it requires $6N - 6$ constraints: $2N$ equality constraints for rows and columns, and $4N - 6$ inequality

Table 1
Mathematical Formulations of Exact Solving Paradigms

Paradigm	Primary Variables	Constraints
SAT	$O(N^2)$ Boolean	$O(N^2)$ to $\Theta(N^3)$ clauses (dependent on AMO strategy)
CP	$O(N)$ Integer	3 Global `AllDifferent` constraints
MIP	$O(N^2)$ Binary	$6N - 6$ Linear equality and inequality constraints

constraints for the diagonals. The model is formulated as a 0-1 integer programming feasibility problem (no objective function) and solved using Gurobi MIP and IBM CPLEX MIP.

Experimental Fairness: To ensure an unbiased comparative baseline, all paradigms are subject to strict experimental fairness parameters. They solve identical N-Queens instance sizes searching for any single valid solution. We enforce single-threaded execution when supported by the solver architecture, apply the same time limits, execute inside the same hardware/software benchmarking environment, and repeat each configuration 5 times. Importantly, all solutions returned by any solver are mathematically verified by a common external validator to guarantee structural correctness. Note that while the formulations are mathematically equivalent, their internal solver representations and search heuristics are entirely distinct.

4. Experimental Setup

The benchmark was conducted on a controlled hardware and software environment to ensure deterministic execution. The experiments were run on macOS 14.0 (Darwin 23.0.0, Apple M2 ARM64 architecture) using Python 3.13.7. The evaluated engines included PySAT Glucose3, OR-Tools CP-SAT (version 9.15.6755), Gurobi Optimizer (version 13.0.3), and IBM ILOG CPLEX / CP Optimizer (version 22.2.0.0).

We evaluated 9 distinct solving methods across 11 instance sizes: $N \in \{4, 8, 16, 20, 31, 32, 40, 44, 50, 64, 100\}$. Each configuration was repeated 5 times, yielding 495 planned official trials. To guarantee fairness, every trial was executed in an isolated subprocess. All methods were configured for single-threaded execution (`workers=1`). For exact baseline solvers exposing random seed and native timeout APIs (Gurobi, CPLEX, CP-SAT), parameters were explicitly set to `random_seed=0` and a 300.0-second internal limit. For Boolean SAT models solved via PySAT Glucose3, the engine does not expose native wall-clock or random seed parameters in this interface; thus, Glucose3 ran under its default deterministic heuristics, bounded by an external 330.0-second subprocess supervisor watchdog that forcefully terminated non-terminating executions.

The primary metric for comparative evaluation across methods is **pipeline total time**, capturing end-to-end execution (model construction, solver loading, search, and independent solution verification). In addition, fine-grained phases were instrumented: for SAT formulations, model creation time (`encoding_time`), formula ingestion into the C++ solver core (`load_time`), and CDCL search (`search_time`) were logged separately, satisfying `solve_time` = `load_time` + `search_time`. The 5 isolated repetitions per configuration measure deterministic runtime and environment variance under a fixed configuration.

Memory consumption was monitored as **sampled process-tree Resident Set Size (RSS) in MiB**, recorded by polling the root process and all recursive children at 0.1-second intervals via `psutil`. Two experimental caveats apply to this metric: periodic sampling may miss transient peak allocations occurring strictly between 0.1s intervals, and summing RSS across parent and child processes may double-count physical pages shared via copy-on-write fork semantics or dynamic system libraries. Runs blocked by proactive Community Edition license constraints (`BLOCKED_LICENSE`) were explicitly flagged to distinguish commercial capacity boundaries from algorithmic intractability.

5. Results and Discussion

Across the 495 planned trials, 430 yielded successfully verified solutions, 20 encountered timeouts, and 45 were blocked by expected MIP license limitations. No solver generated an invalid placement.

5.1. Structural Comparison of SAT Encodings

As instance size grows, the structural disparities of CNF generation become stark. At $N = 100$, all encodings instantiate identical 10,000 primary binary variables but vary widely in auxiliary metrics. The Pairwise encoding

Table 2
Experimental Configuration

Parameter	Value
Hardware	Apple M2 (ARM64, 8 cores, 16 GiB unified memory)
OS	macOS 14.0 (Darwin 23.0.0)
Python Version	3.13.7
SAT Solver Backend	PySAT Glucose3 (v1.9.dev15)
SAT Phase Policy	`solver_default`
CP Solvers	OR-Tools CP-SAT (v9.15.6755), IBM CP Optimizer (v22.2.0.0)
MIP Solvers	Gurobi Optimizer (v13.0.3), IBM CPLEX MIP (v22.2.0.0)
Time Limits	300.0s (Internal exact solvers), 330.0s (External supervisor watchdog)
Workers	1 (Single-threaded execution)
Random Seed	0 (Exact solvers); default deterministic heuristics (PySAT Glucose3)
Repetitions	5 isolated subprocesses per configuration
Memory Measurement	Sampled process-tree RSS in MiB (0.1s interval)

requires zero auxiliary variables but scales poorly, producing 1,646,900 clauses. The Sequential Counter encoding generates the second-fewest clauses (117,812) at the cost of the highest auxiliary count (39,402). The Product encoding achieves the most compact configuration, producing 116,672 clauses and 9,492 auxiliary variables. Binary and Commander encodings balance these metrics.

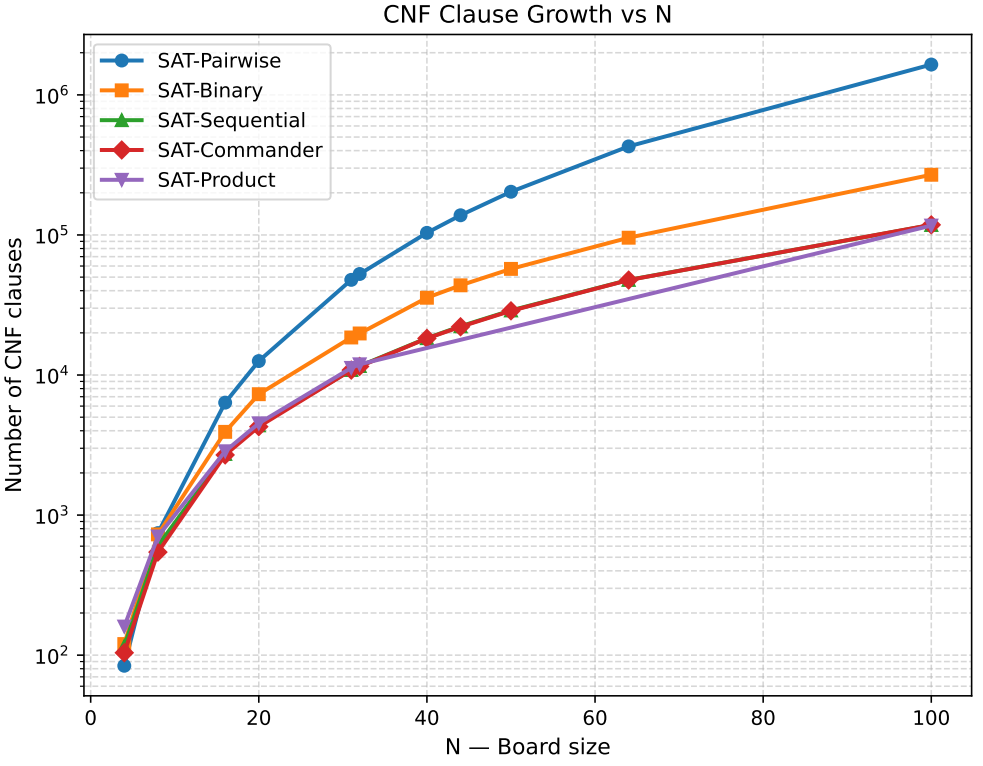


Figure 1: SAT CNF Clause Growth

5.2. Runtime Performance of SAT Encodings

The resolution time for SAT formulations proves that clause count is an inadequate predictor of performance.

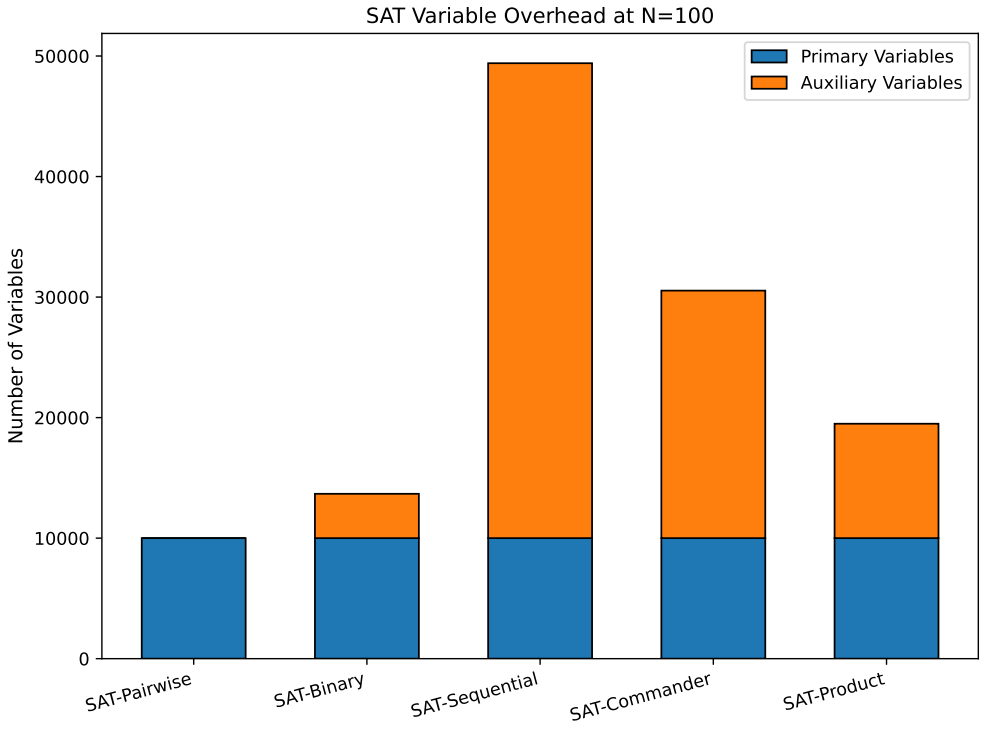


Figure 2: SAT Auxiliary Variable Overhead

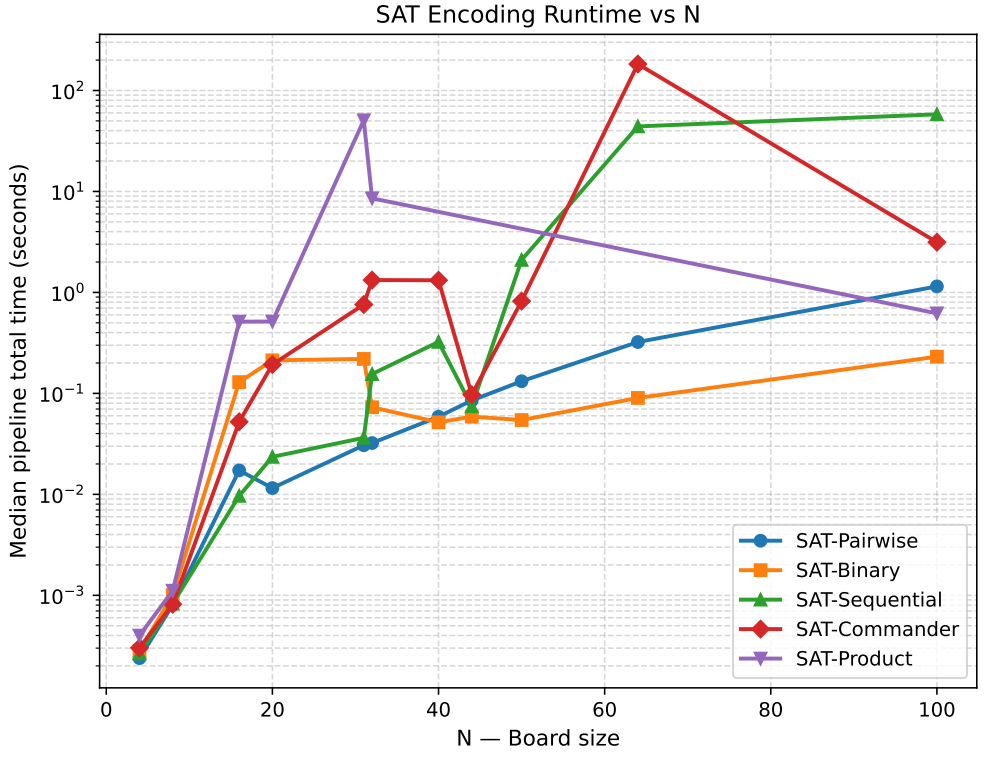


Figure 3: SAT Runtime vs N

Table 3
Fine-grained execution phase decomposition and Glucose3 CDCL metrics at $N = 100$ (medians across 5 runs).

Encoding	Pipeline (s)	Encoding (s)	Load (s)	Search (s)	Solve (s)	Conflicts	Decisions	Propagations	Restarts
Pairwise	1.148	0.452	0.446	**0.021**	0.466	**277**	**8,211**	**38,180**	**2**
Binary	**0.231**	0.079	0.085	0.028	**0.113**	328	9,588	117,046	4
Product	0.620	**0.040**	0.041	0.518	0.558	4,538	21,956	1,702,119	6
Commander	3.143	0.055	**0.037**	3.028	3.064	18,696	50,856	7,185,228	11
Sequential	58.078	0.043	0.038	57.997	58.026	139,057	172,554	70,674,327	6

At $N = 100$, the SAT-Binary encoding achieved the fastest median pipeline runtime of 0.231 seconds. In stark contrast, the SAT-Sequential encoding, despite producing minimal clauses, recorded the slowest median runtime of 58.077 seconds. The clause-heavy SAT-Pairwise encoding successfully completed the search in a median pipeline time of 1.148 seconds.

To understand the underlying mechanisms driving these runtime disparities, Table 3 decomposes the execution phases and reports the internal Glucose3 CDCL search counters recorded at $N = 100$. Note that because each column reports the independent sample median across 5 runs, component medians do not strictly sum to the total median.

This granular breakdown reveals a critical empirical finding: **SAT-Pairwise actually achieves the fastest pure CDCL search time (0.021 s)** and the lowest search effort across all solvers (277 conflicts and 38,180 unit propagations, compared to 328 conflicts and 117,046 propagations for Binary). However, Pairwise generates $1,646,900$ clauses; constructing this massive formula in Python takes 0.452 s (`encoding_time`) and ingesting the clause array into the PySAT Glucose3 C++ engine takes 0.446 s (`load_time`), accumulating ≈ 0.90 s of model preparation overhead before search even commences.

Conversely, SAT-Binary produces only $269,024$ clauses, reducing combined encoding and loading overhead to 0.164 s. Because its pure search time is exceptionally competitive (0.028 s), Binary achieves the best end-to-end pipeline time (0.231 s).

At the opposite extreme, SAT-Sequential exhibits negligible encoding and loading costs (0.043 s and 0.038 s), but its CDCL search phase degenerates severely (57.997 s search time, 139,057 conflicts, and over 70.6 million unit propagations). In Sinz's sequential counter formulation, long linear implications $s_{i-1} \implies s_i$ and $\neg x_i \vee \neg s_{i-1}$ create deep dependency chains. Under Glucose3's default variable activity and phase-saving heuristics, the solver repeatedly traverses these linear counter chains, causing massive thrashing and slow backjumping compared to the logarithmic bitwise branching enabled by Binary.

5.3. Non-monotonic Search Behavior of Product Encoding

While SAT-Product reliably solved small sizes and successfully resolved $N = 100$ (median 0.620s), it completely failed at intermediate sizes ($N \in \{40, 44, 50, 64\}$), resulting in a timeout for all 5 repetitions.

This non-monotonic scaling underscores the sensitivity of CDCL solvers to heuristic traps. Under Glucose3's `solver_default` policy, the specific auxiliary-variable structure of Product at intermediate bounds may force the search into pathological branching spaces. However, these are potential contributing factors; analyzing the exact causes of this behavior requires further experimental study of internal solver traces and variable ordering.

5.4. Performance of Exact Solvers

Native Constraint Programming (CP) architectures demonstrated highly favorable scaling characteristics.

Both IBM CP Optimizer and OR-Tools CP-SAT maintained 100% reliability up to $N = 100$. The CP Optimizer solver established itself as remarkably efficient. The MIP solvers successfully solved bounds within their Community Edition license constraints.

5.5. Cross-Paradigm Comparison at N=100

At $N = 100$, seven methods successfully executed. The IBM CP Optimizer achieved the lowest median runtime of 0.153 seconds.

Remarkably, the booleanized SAT-Binary formulation tracked incredibly close at 0.231 seconds, outpacing OR-Tools CP-SAT (0.808 seconds).

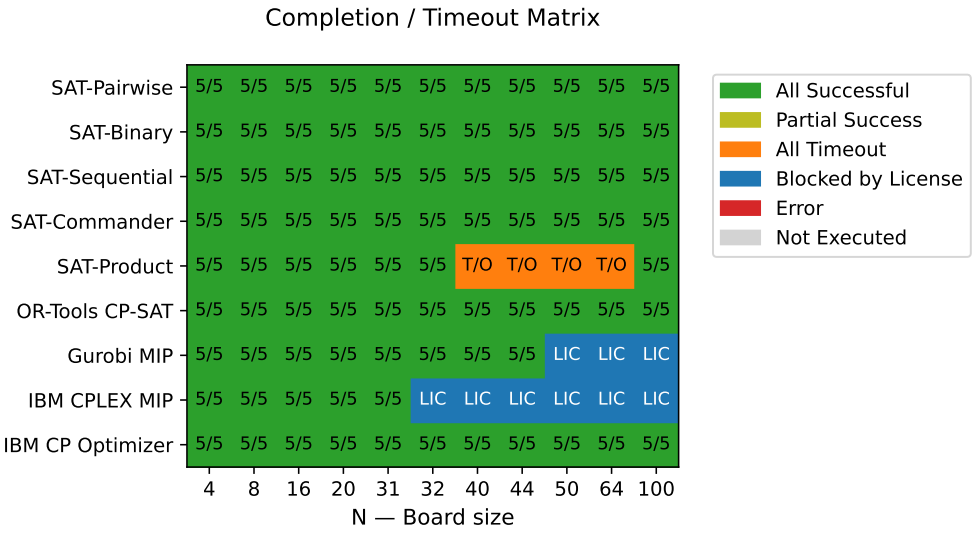


Figure 4: Completion Matrix

Table 4
SAT Encoding Structural Complexity at N=100

Encoding	Primary Variables	Auxiliary Variables	Clauses
Pairwise	10,000	0	1,646,900
Binary	10,000	3,678	269,024
Sequential	10,000	39,402	117,812
Commander	10,000	20,536	118,092
Product	10,000	9,492	116,672

Table 5
Cross-Paradigm Comparison at N=20

Method	Success Rate	Median Runtime (s)
Gurobi MIP	5/5	0.0068
SAT-Pairwise	5/5	0.0115
OR-Tools CP-SAT	5/5	0.0160
IBM CP Optimizer	5/5	0.0205
SAT-Sequential	5/5	0.0235
IBM CPLEX MIP	5/5	0.0309
SAT-Commander	5/5	0.1928
SAT-Binary	5/5	0.2124
SAT-Product	5/5	0.5150

6. Threats to Validity

Several factors condition the interpretation of our findings. First, while standard N-Queens on an empty board admits constructive $O(N)$ solutions, we employ it strictly as a canonical benchmark for comparing how general-purpose reasoning architectures formulate, propagate, and resolve expanding symmetric constraints. These results should not be conflated with the NP-complete N-Queens Completion problem (Gent et al., 2017). Second, to establish an untuned algorithmic baseline, all Boolean SAT encodings were executed under Glucose3's default phase-saving policy without dynamic random seed manipulation; modifying branching heuristics could radically alter search trajectories. Thus, the timeouts observed in SAT-Product and SAT-Sequential reflect specific heuristic-encoding interactions under

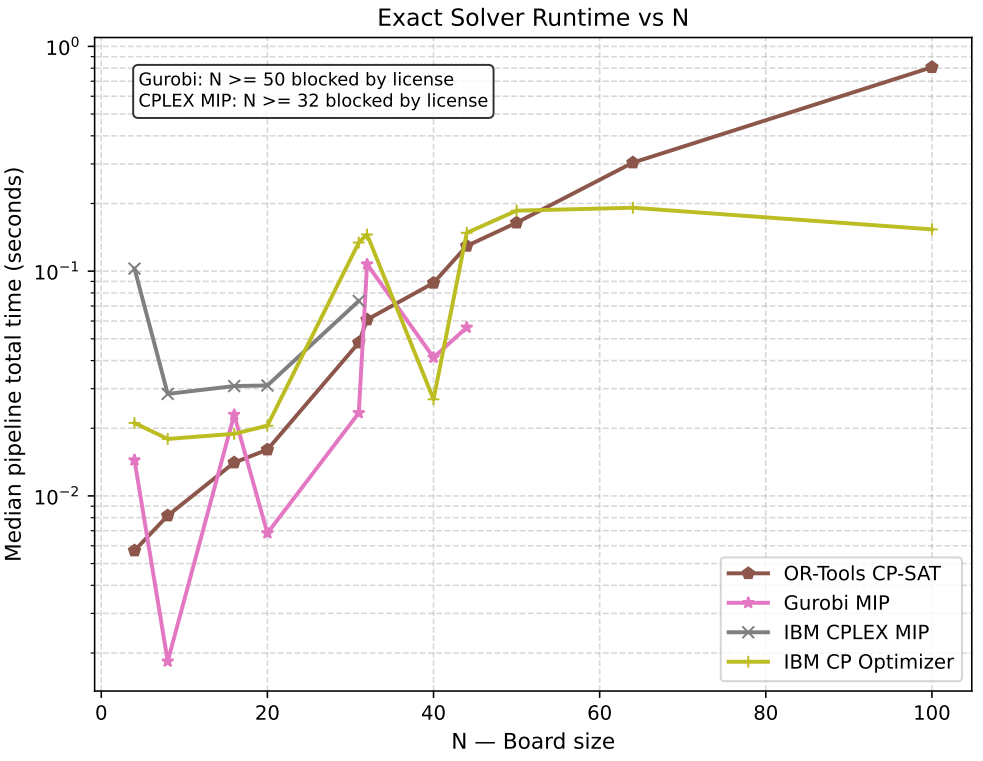


Figure 5: Exact Solver Runtime vs N

Table 6
Cross-Paradigm Comparison at N=100

Method	Success Rate	Median Runtime (s)
IBM CP Optimizer	5/5	0.153
SAT-Binary	5/5	0.231
SAT-Product	5/5	0.620
OR-Tools CP-SAT	5/5	0.808
SAT-Pairwise	5/5	1.148
SAT-Commander	5/5	3.143
SAT-Sequential	5/5	58.077
Gurobi MIP	N/A	BLOCKED_LICENSE
IBM CPLEX MIP	N/A	BLOCKED_LICENSE

this baseline rather than absolute intractability. Third, while our primary metric (pipeline total time) aggregates CNF construction, solver loading, search, and validation, formula generation overhead differs markedly across encodings: Pairwise incurs $\Theta(N^3)$ clauses across the board, Binary requires $\Theta(N^2 \log N)$ total clauses, whereas Sequential, Commander ($k = 3$), and Product generate $\Theta(N^2)$ total clauses. Consequently, SAT approaches carry formulation overheads absent in native CP models, though hybrid engines like OR-Tools CP-SAT internally generate Boolean conflict clauses via Lazy Clause Generation. Furthermore, memory consumption was measured as sampled process-tree Resident Set Size (RSS) in MiB, which is subject to sampling discretization (potentially missing inter-sample spikes) and may double-count pages shared across parent and child processes.

The supplementary large-scale experiment at $N = 200$ introduces additional validity considerations. Execution was limited to five isolated repetitions per method under a deterministic setting, which measures environment and system runtime variance but does not capture the full distribution of randomized search trajectories. Crucially, the

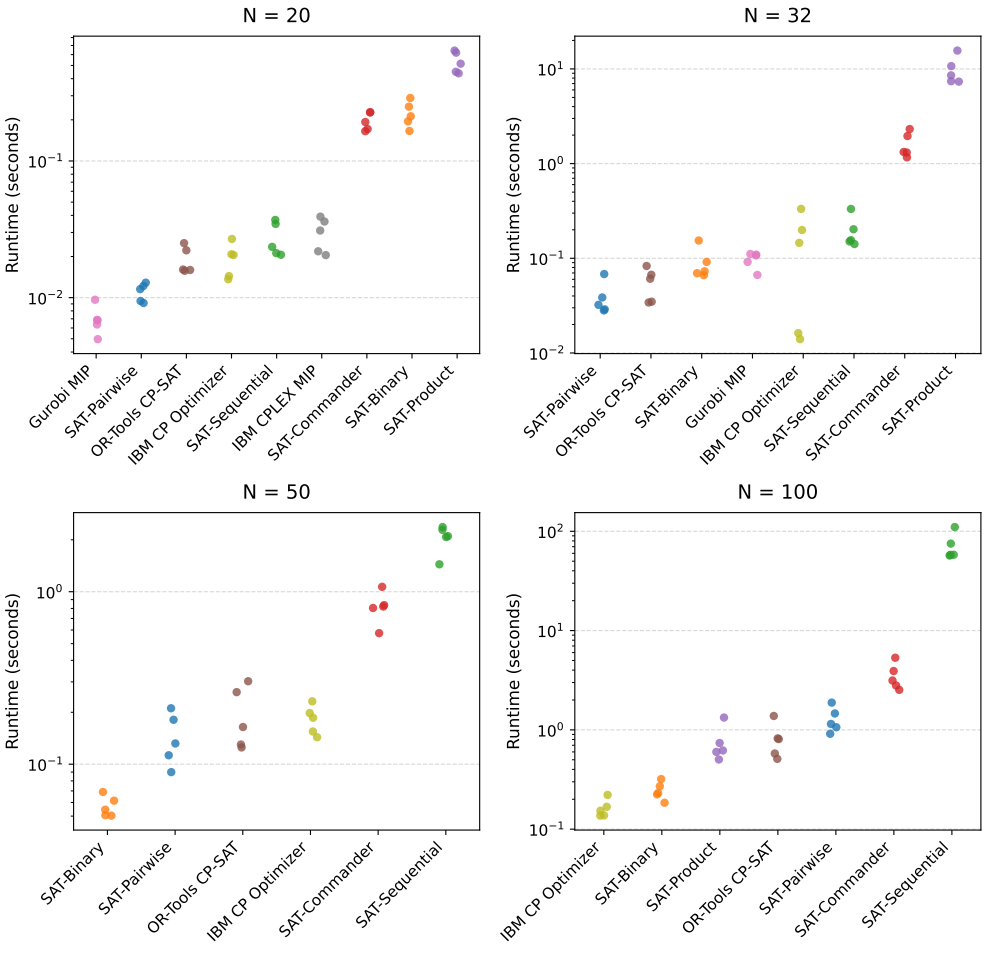


Figure 6: Runtime Variability

exclusion of SAT-Pairwise at $N = 200$ was enforced proactively by a preflight safety policy. Specifically, the preflight protocol established an effective available memory cap of 798.34 MiB (dynamically computed from host free memory minus a safety reserve) bounded by a maximum policy ceiling of 4,096 MiB. The analytical structural projection for SAT-Pairwise at $N = 200$ was 4,175.18 MiB (for over 13.2 million clauses), thereby exceeding both the active headroom cap and the policy ceiling, triggering proactive exclusion (`NOT_RUN_RESOURCE_POLICY`) to preserve host system stability rather than an unmonitored runtime crash. Similarly, the IBM CP Optimizer's `LICENSE_ERROR` was triggered by a commercial Community Edition search-space bound (2^{1000}) rather than algorithmic exhaustion. Finally, evaluating a supplementary point at $N = 200$ demonstrates extreme-scale feasibility but does not replace continuous interpolation across intermediate problem sizes.

7. Supplementary Large-Scale Experiment

To assess the scalability of exact solving paradigms beyond the $N = 100$ boundary, an independent, supplementary experiment was conducted at $N = 200$ (representing a board size of 200×200 with 40,000 primary Boolean variables). A strict memory guard policy was introduced to prevent hardware exhaustion. Table 8 presents the solver outcomes based on 5 isolated repetitions per method.

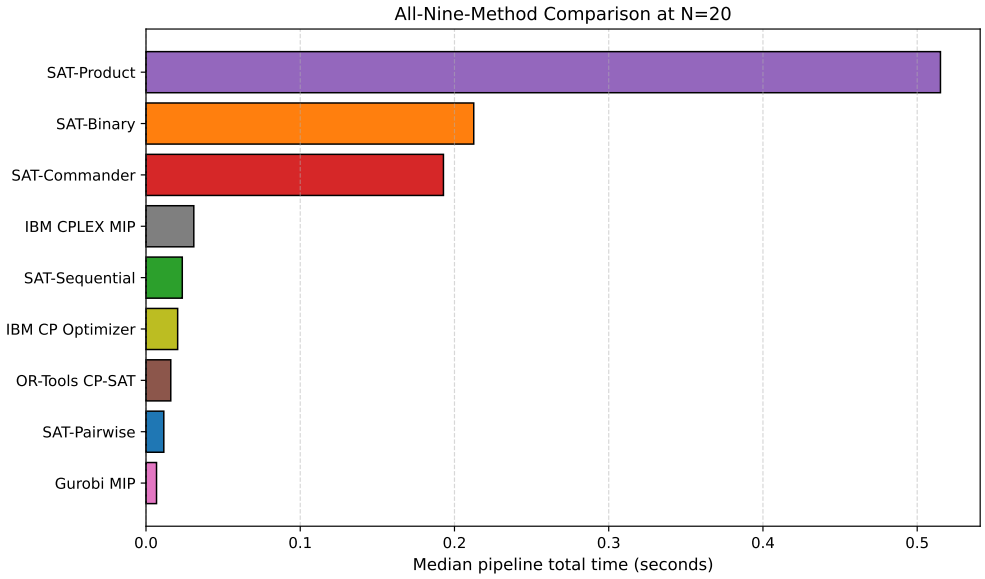


Figure 7: All Nine Methods at N=20

Table 7
Completion Reliability Across All 495 Trials

Method	Successful Runs	Timeout Runs	License Blocked Runs
SAT-Pairwise	55	0	0
SAT-Binary	55	0	0
SAT-Sequential	55	0	0
SAT-Commander	55	0	0
SAT-Product	35	20	0
OR-Tools CP-SAT	55	0	0
IBM CP Optimizer	55	0	0
Gurobi MIP	40	0	15
IBM CPLEX MIP	25	0	30
Total	**430**	**20**	**45**

The scaling results fundamentally underscore the intersection of mathematical formulation and hardware capacity. The SAT-Pairwise formulation, which creates $\Theta(N^3)$ clauses, generated an estimated 13.2 million clauses $(13,253,800)$ at $N = 200$ requiring an estimated $4,175.18$ MiB of continuous memory allocation. Because this structural projection exceeded both the active preflight memory headroom cap of 798.34 MiB and the maximum policy ceiling of $4,096$ MiB, the preflight guard proactively refused execution (`NOT_RUN_RESOURCE_POLICY`). SAT-Sequential experienced universal timeouts, aligning with its severe performance degradation observed at lower values of N. Moreover, the IBM CP Optimizer encountered a `LICENSE_ERROR` triggered by its Community Edition search space limit (2^{1000} combinations), which is easily breached by the $N = 200$ variables.

As shown in Figure 8, the methods that successfully scaled (SAT-Commander, SAT-Product, SAT-Binary, and OR-Tools CP-SAT) maintained manageable memory footprints due to their structural compactness. Among the four configurations that completed successfully at $N = 200$ in this benchmarking environment, SAT-Commander recorded the lowest median pipeline runtime (0.441 s) and the lowest peak sampled process-tree RSS (146.2 MiB), outperforming SAT-Product (0.627 s, 165.1 MiB), SAT-Binary (0.815 s, 288.9 MiB), and OR-Tools CP-SAT (3.672 s, 259.7 MiB). Interestingly, the SAT-Product encoding, which previously displayed non-monotonic failures and pathological timeouts at $N = 44$ and $N = 64$, effectively recovered at $N = 200$ to achieve a median runtime of 0.627 seconds. Furthermore, SAT-Commander exhibited an inverted runtime scaling ratio (faster at $N = 200$ than

Table 8
Large-scale experimental outcomes at $N = 200$. Medians calculated over successful trials only.

Method	Status	Success/5	Median Time (s)	Peak RAM (MiB)
SAT-Binary	SAT	5	0.815	288.9
SAT-Pairwise	NOT_RUN_RESOURCE_POLICY	0	N/A	N/A
SAT-Sequential	TIMEOUT	0	N/A	649.5
SAT-Commander	SAT	5	0.441	146.2
SAT-Product	SAT	5	0.627	165.1
OR-Tools CP-SAT	SAT	5	3.672	259.7
IBM CP Optimizer	LICENSE_ERROR	0	N/A	87.1
Gurobi MIP	BLOCKED_LICENSE	0	N/A	N/A
IBM CPLEX MIP	BLOCKED_LICENSE	0	N/A	N/A

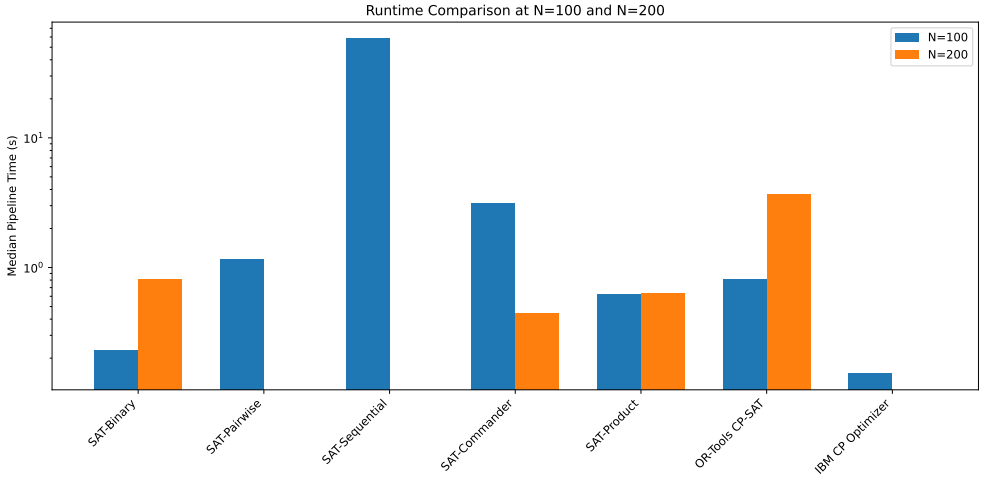


Figure 8: Median pipeline time comparison between $N = 100$ and $N = 200$ for successful methods.

$N = 100$). These observations demonstrate that specific variable partitioning strategies can favorably align with the default phase-saving heuristics of modern CDCL architectures on large symmetric grids, although general conclusions across diverse solvers require broader theoretical and empirical validation.

Figure 9 provides a holistic view of the execution outcomes at $N = 200$. While native solvers were predominantly bounded by license restrictions, SAT models demonstrated a wider variance of outcomes encompassing memory exclusions (`NOT_RUN_RESOURCE_POLICY`), timeouts, and successful resolutions.

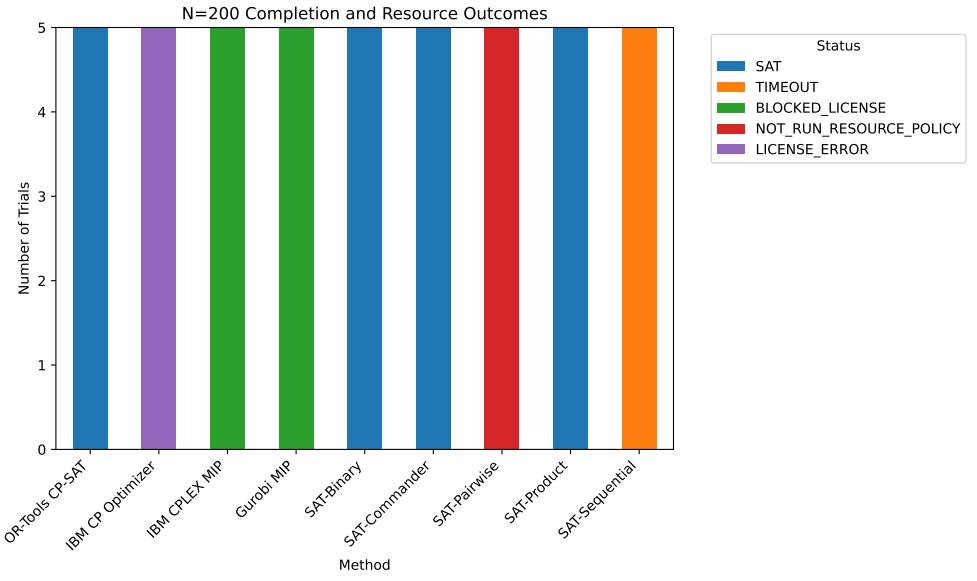


Figure 9: Distribution of completion statuses and resource limitations at $N = 200$.

8. Conclusion

This research provides a strictly controlled empirical evaluation of three major exact solving paradigms—Boolean SAT, Constraint Programming, and Mixed Integer Programming—on the N-Queens problem. Through the execution of 495 scheduled benchmark trials (430 successful completions) up to $N = 100$, we observed that theoretical SAT structural compactness is a poor predictor of practical solving efficiency. Massive clause models (Pairwise) frequently outpaced compact-clause implementations (Sequential Counter). Under default heuristic settings, the SAT-Binary encoding emerged as the most efficient boolean model, completing $N = 100$ in 0.231 seconds.

At $N = 200$, the limitations of both hardware and software environments became evident. The SAT-Pairwise encoding was proactively excluded due to an estimated memory footprint exceeding hardware limits, while IBM CP Optimizer encountered a license-imposed search space restriction. Among the four configurations that completed successfully at $N = 200$ in this benchmarking environment, SAT-Commander achieved the lowest median pipeline resolution time (0.441 seconds) and the lowest peak sampled process-tree RSS (146.2 MiB), outpacing SAT-Product (0.627 s, 165.1 MiB), SAT-Binary (0.815 s, 288.9 MiB), and OR-Tools CP-SAT (3.672 seconds, 259.7 MiB), demonstrating that optimal Boolean partitioning can remain exceptionally competitive even at massive scales. However, native Constraint Programming implementations handled the large state spaces efficiently. The IBM CP Optimizer recorded the fastest runtime overall, resolving $N = 100$ in 0.153 seconds by circumventing the massive clause-loading overheads required by SAT translation. Nevertheless, CP frameworks still suffer from exponential backtracks and do not universally eliminate search limitations. Furthermore, we documented severe non-monotonic search pathologies in the SAT-Product encoding, underscoring the extreme sensitivity of CDCL solvers to variable layout.

Future investigations should focus on phase-policy ablation studies to isolate the mechanisms triggering SAT search pathologies, as well as evaluating these structural encodings on a broader suite of combinatorial benchmarks beyond placement constraints.

A. Formal Derivations of Exact Structural Complexities

This appendix provides complete, self-contained algebraic derivations for the exact clause and auxiliary variable counts generated by the Boolean SAT encodings on an $N \times N$ chessboard for any integer $N \geq 2$.

A.1. Board Geometric Decomposition and Group Cardinalities

The standard N-Queens problem on an $N \times N$ grid is partitioned into four families of cardinality constraints:

1. **Row Constraints:** Exactly N rows, each containing a group of $m = N$ primary variables requiring Exactly-One (1 ALO clause + 1 AMO constraint per row).
2. **Column Constraints:** Exactly N columns, each containing a group of $m = N$ primary variables requiring Exactly-One (1 ALO clause + 1 AMO constraint per column).
3. **Main Diagonals ($i - j = d$):** The diagonal index ranges over $d \in \{-(N-1), \dots, N-1\}$. The two corner diagonals ($d = \pm(N-1)$) contain a single cell ($m = 1$) and are trivially non-conflicting. For non-trivial diagonals requiring At-Most-One ($m \geq 2$):
   - The main diagonal ($d = 0$) has length $m = N$.
   - For each integer length $m \in \{2, \dots, N-1\}$, there are exactly 2 symmetric diagonals of length m (corresponding to $d = N - m$ and $d = -(N-m)$).
4. **Anti-Diagonals ($i + j = s$):** The anti-diagonal index ranges over $s \in \{0, \dots, 2N-2\}$. The two corner cells ($s = 0$ and $s = 2N-2$) contain $m = 1$. For non-trivial anti-diagonals ($m \geq 2$):
   - The central anti-diagonal ($s = N-1$) has length $m = N$.
   - For each integer length $m \in \{2, \dots, N-1\}$, there are exactly 2 symmetric anti-diagonals of length m.

Summing across both diagonal families, the board contains:

- Exactly 2 diagonals of length N.
- Exactly 4 diagonals of length m for each $m \in \{2, \dots, N-1\}$.

The total number of non-trivial diagonal groups requiring AMO is $2 + 4(N-2) = 4N - 6$. Combined with the $2N$ row and column groups, the entire board comprises $6N - 6$ constraint groups for $N \geq 2$.

A.2. Exact Clause Count for Pairwise AMO Encoding

Theorem 1. *For the N-Queens problem with $N \geq 2$, the Pairwise (Binomial) encoding generates zero auxiliary variables and an exact total clause count of:*

$$C_{\text{pairwise}}(N) = 2N + N^2(N-1) + \frac{N(N-1)(2N-1)}{3} \tag{20}$$

Proof. The total clause count consists of $2N$ At-Least-One clauses (one for each row and column) plus the sum of Pairwise AMO clauses across all groups. A group of size m generates $\binom{m}{2} = \frac{m(m-1)}{2}$ exclusion clauses.

1. **Rows and Columns:** There are $2N$ groups of size N, yielding:

$$C_{\text{row,col}}^{\text{AMO}} = 2N \cdot \frac{N(N-1)}{2} = N^2(N-1) \tag{21}$$

2. **Diagonals:** The diagonal groups comprise 2 groups of length N and 4 groups of each length $m \in \{2, \dots, N-1\}$:

$$\begin{aligned} C_{\text{diag}}^{\text{AMO}} &= 2 \cdot \frac{N(N-1)}{2} + \sum_{m=2}^{N-1} 4 \cdot \frac{m(m-1)}{2} \\ &= N(N-1) + 2 \sum_{m=2}^{N-1} (m^2 - m) \end{aligned} \tag{22}$$

Applying the standard summation identities $\sum_{k=1}^{K} k = \frac{K(K+1)}{2}$ and $\sum_{k=1}^{K} k^2 = \frac{K(K+1)(2K+1)}{6}$ with upper bound $K = N-1$:

$$\sum_{m=2}^{N-1} (m^2 - m) = \sum_{m=1}^{N-1} (m^2 - m)$$

$$= \frac{(N-1)N(2N-1)}{6} - \frac{(N-1)N}{2}$$
$$= \frac{N(N-1)(2N-4)}{6} = \frac{N(N-1)(N-2)}{3} \tag{23}$$

Substituting this back into $C_{\text{diag}}^{\text{AMO}}$:

$$C_{\text{diag}}^{\text{AMO}} = N(N-1) + \frac{2N(N-1)(N-2)}{3}$$
$$= \frac{N(N-1)[3+2(N-2)]}{3} = \frac{N(N-1)(2N-1)}{3} \tag{24}$$

Summing the $2N$ ALO clauses, the row/column AMO clauses, and the diagonal AMO clauses yields:

$$C_{\text{pairwise}}(N) = 2N + N^2(N-1) + \frac{N(N-1)(2N-1)}{3} \tag{25}$$

Expanding this polynomial yields $C_{\text{pairwise}}(N) = \frac{5}{3}N^3 - 2N^2 + \frac{7}{3}N \in \Theta(N^3)$, which proves the theorem. ■

A.3. Exact Complexity for Sinz Sequential Counter AMO Encoding

Theorem 2. *For the N-Queens problem with $N \geq 2$, the Sinz Sequential Counter encoding generates exact totals of:*

$$A_{\text{sequential}}(N) = 4N^2 - 6N + 2 \tag{26}$$
$$C_{\text{sequential}}(N) = 12N^2 - 22N + 12 \tag{27}$$

auxiliary variables and clauses, respectively.

Proof. In Sinz's sequential counter encoding for an At-Most-One constraint over $m \geq 2$ variables, exactly $m-1$ auxiliary variables are instantiated, and exactly $3m-4$ clauses are generated (comprising 1 initial implication clause, $3(m-2)$ transition and exclusion clauses, and 1 terminal exclusion clause).

Auxiliary Variables ($A_{\text{sequential}}$):

1. **Rows and Columns:** $2N$ groups of size N contribute $2N(N-1)$ auxiliary variables.
2. **Diagonals:** The diagonal groups contribute:

$$A_{\text{diag}} = 2(N-1) + \sum_{m=2}^{N-1} 4(m-1)$$
$$= 2(N-1) + 4\sum_{j=1}^{N-2} j$$
$$= 2(N-1) + 4 \cdot \frac{(N-2)(N-1)}{2}$$
$$= 2(N-1) + 2(N-2)(N-1) = 2(N-1)^2 \tag{28}$$

Summing the row/column and diagonal auxiliaries:

$$A_{\text{sequential}}(N) = 2N(N-1) + 2(N-1)^2$$
$$= 2(N-1)(N+N-1) = 2(N-1)(2N-1)$$
$$= 4N^2 - 6N + 2 \tag{29}$$

Clauses ($C_{\text{sequential}}$): The clause total includes $2N$ ALO clauses plus the AMO clauses across all groups ($3m-4$ clauses per group of size m):

1. **Rows and Columns:** $2N(3N-4) = 6N^2 - 8N$.

Table 9
Exact theoretical vs. generated structural counts across instance sizes.

N	C_{pairwise} (Formula)	C_{pairwise} (Generated)	$C_{\text{sequential}}$ (Formula)	$C_{\text{sequential}}$ (Generated)
4	84	84	116	116
8	744	744	604	604
16	6,248	6,248	2,732	2,732
20	12,380	12,380	4,372	4,372
32	51,760	51,760	11,596	11,596
50	203,770	203,770	28,912	28,912
64	424,848	424,848	47,820	47,820
100	1,646,900	1,646,900	117,812	117,812
200	13,253,800	13,253,800	475,612	475,612

2. **Diagonals:**

$$\begin{aligned} C_{\text{diag}}^{\text{AMO}} &= 2(3N-4) + \sum_{m=2}^{N-1} 4(3m-4) \\ &= (6N-8) + 12\sum_{m=2}^{N-1} m - 16(N-2) \end{aligned} \tag{30}$$

Evaluating the summation $\sum_{m=2}^{N-1} m = \frac{(N-1)N}{2} - 1 = \frac{N^2-N-2}{2}$:

$$\begin{aligned} C_{\text{diag}}^{\text{AMO}} &= 6N - 8 + 6(N^2 - N - 2) - 16N + 32 \\ &= 6N^2 + (6 - 6 - 16)N + (-8 - 12 + 32) \\ &= 6N^2 - 16N + 12 \end{aligned} \tag{31}$$

Summing the ALO clauses, row/column AMO clauses, and diagonal AMO clauses:

$$\begin{aligned} C_{\text{sequential}}(N) &= 2N + (6N^2 - 8N) + (6N^2 - 16N + 12) \\ &= 12N^2 - 22N + 12 \end{aligned} \tag{32}$$

Both $A_{\text{sequential}}(N)$ and $C_{\text{sequential}}(N)$ are quadratic in N ($\Theta(N^2)$), which proves the theorem. ■

A.4. Empirical Verification of Closed-Form Expressions

Table 9 validates the closed-form expressions against the values produced by the benchmark model generator across representative instance sizes.

Data Availability

The complete source code, solver implementations, test suites, and benchmarking reproduction scripts are openly available in the GitHub repository: `https://github.com/anhson1207/NQueens-SAT`. All experimental datasets (including raw benchmark JSONL logs, processed CSV matrices, environment metadata, and statistical figures) are tracked directly within the repository under the `results/` directory:

- **Primary Full Benchmark ($N \leq 100$):** Contains 513 records total (495 planned official trials and 18 warmup runs) across 9 solving methods at `results/raw/benchmark_nqueens_primary_full.jsonl`, with processed summaries and environment telemetry at `results/metadata/`.
- **Supplementary Large-Scale Experiment ($N = 200$):** Contains 45 scheduled trials (20 SAT, 5 TIMEOUT, 5 LICENSE_ERROR, 10 BLOCKED_LICENSE, 5 NOT_RUN_RESOURCE_POLICY) recorded at `results/raw/benchmark_nqueens_large_scale_n200.jsonl`, accompanied by preflight environment telemetry and cryptographic source tree fingerprints at `results/metadata/benchmark_nqueens_large_scale_n200_preflight.json`.

- **Historical Provenance:** Earlier development logs include 269 legacy trial records gathered prior to schema fingerprint automation, which were independently validated via configuration and outcome consistency without retroactive hash alteration.

References

Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers, in: International Joint Conference on Artificial Intelligence (IJCAI).

Bell, J., Stevens, B., 2009. A survey of known results and research areas for n-queens. Discrete Mathematics 309, 1–31.

Chen, J., 2010. A new SAT encoding of the at-most-one constraint, in: Tenth International Workshop on Constraint Modelling and Reformulation (ModRef 2010), pp. 1–9.

Cook, S.A., 1971. The complexity of theorem-proving procedures, in: Proceedings of the third annual ACM symposium on Theory of computing, pp. 151–158.

Duong, S., 2024. nQueens-SAT: Empirical comparison of N-Queens encodings and baseline solvers. https://github.com/Duongsuny/nQueens-SAT.

Frisch, A.M., Giannaros, P.A., 2010. SAT encodings of the at-most-k constraint: Some old, some new, some fast, some slow, in: Ninth International Workshop on Constraint Modelling and Reformulation (ModRef 2010).

Gent, I.P., Jefferson, C., Nightingale, P., 2017. Complexity of n-queens completion. Journal of Artificial Intelligence Research 59, 815–848.

Gomes, C.P., Kautz, H., Sabharwal, A., Selman, B., 2008. Satisfiability Solvers. Elsevier.

Klieber, W., Kwon, G., 2007. Efficient CNF encoding for selecting 1 from N objects, in: Proceedings of the 2007 International Workshop on Constraints in Formal Verification (CFV'07), pp. 1–10.

Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning sat solvers, in: Handbook of Satisfiability, IOS Press. pp. 131–153.

Perron, L., Furnon, V., 2019. OR-Tools. URL: https://developers.google.com/optimization.

Régin, J.C., 1994. A filtering algorithm for constraints of difference in csps, in: National Conference on Artificial Intelligence (AAAI), pp. 362–367.

Rossi, F., Van Beek, P., Walsh, T., 2006. Handbook of Constraint Programming. Elsevier.

Russell, S., Norvig, P., 2010. Artificial Intelligence: A Modern Approach. 3rd ed., Prentice Hall.

Sinz, C., 2005. Towards an optimal cnf encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP), Springer. pp. 827–831.

Wolsey, L.A., 2020. Integer Programming. John Wiley & Sons.